

4주차 과제

패킹과 pe구조에 관한 분석과 실습

패킹 정의와 구조

패킹의 정의: 패킹(Packing)은 실행 파일의 원래 코드와 데이터를 압축·암호화·변형해서 파일 안에 숨겨 두고, 실행 시점에 이를 메모리에서 복원한 뒤 실행하도록 만드는 기술이다.

구조를 단순하게 묘사하면

원본 프로그램

[PE Header]

[원래 코드]

[원래 데이터]

[Import Table]

에서

패킹된 프로그램

[변형된 PE Header]

[복원 코드: Unpacking Stub]

[압축·암호화된 원래 프로그램]

[축소되거나 변형된 Import Table]

로 바뀐다.

즉, 패킹된 파일은 처음부터 원래 프로그램을 실행하지 않는다.

먼저 패커가 추가한 작은 복원 프로그램이 실행된다. 이 복원 프로그램을 보통 unpacking stub, loader stub, decompression stub, packer stub와 같이 부른다.

패커의 사용 목적 및 중요 분석 요소

과거에는 실행 파일의 크기를 줄이는 게 제 1의 목표였다. 원본 exe파일의 크기를 줄이고 실행시 다시 원래 크기만큼 메모리를 차지하는 방식으로 작동해서 메모리를 절약할 수 있었다

현대에 들어와서는 그 목적보다는 보안을 위해서 패킹이 더 자주 사용이 되기 시작했다.

```
if (license_is_valid()) {  
  
    run_program();  
  
}
```

이런 코드를 패킹없이 ida에 올리면 상당히 그 구조를 완전히 동일하진 않겠지만 유사하게 복원해 낼 수 있을 것이다. 하지만 만약 패킹을 거친 채로 이 파일을 ida같은 정적 분석 도구에 올린다면 암호화된 상태이기 때문에 그 구조를 파악하는 게 어렵고 실제 메모리에 올려서 동적 디버거로 동작시켜야지 그 작동원리를 파악할 수 있을 것이다.

따라서 실제로 악성코드들도 패킹을 많이 사용한다. 물론 패킹을 한다고 모두 다 악성 프로그램은 아니지만 대부분의 악성코드들은 패킹을 한다 그래서 만약 어떤 파일을 실행시키기전에 의심스러운 경우 ida로 먼저 열어봐서 깨지는 지 여부를 한 번 확인해 보는 것도 나쁘지 않은 방법일 수 있다.

결론은 보안 분석에서는 패킹 여부를 중요하게 본다.

또한 패킹은 압축,암호화,난독화,PE 구조 변형,Import 숨기기,실행 시 메모리 복원 중 하나 이상을 결합한다.

패킹 분석에서 가장 중요한 두가지 요소에 대해서 알아보자. 패킹 분석에서 가장 중요한 요소 두가지가 있는데 그 중 첫 번째 EP: Entry Point는 PE 파일의 헤더에는 프로그램 실행을 어디서 시작할지 나타내는 값으로 정확히는 Optional Header의 AddressOfEntryPoint로 일반적으로 패킹이 되지 않은 프로그램에서는 주로 컴파일러가 만든 초기화 코드 또는 프로그램 시작 코드가 EP에 위치한다.

하지만 패킹이 된 프로그램에서는 이러한 EP는 통 원래 프로그램이 아니라 패커 stub을 가리키도록 바뀐다. 따라서 패킹이 된 프로그램에서 가장 먼저 찾아야 하는 것은 패킹 stub가 끝난 후에 원래 프로그램 코드가 시작되는 OEP다.

이에 대해서 좀 더 깊은 이해를 위해서 패킹된 프로그램이 어떻게 실행되는 지를 구체적으로 한 번 파악해 보자

패킹된 프로그램의 실행절차

1. windows는 패킹 여부를 판단해서 패킹된 파일을 풀어주지 않는다. 단지 패킹 여부와 상관없이 Windows Loader는 PE 헤더를 보고 일반적인 PE 파일처럼 메모리에 매핑할 뿐이다. 그 다음에는 PE 헤더 검사,섹션 매핑,이미지 베이스 결정,기본적인 import 처리,relocation 처리,실행 권한 설정,Entry Point 호출등을 한다. 그런데 여기서 entry point는 패킹 stub일 뿐이다.
- 2.그 다음 entry point로 진입하면 패커 stub는 압축된 데이터 위치 찾기,복원할 메모리 확보,복호화 또는 압축 해제,원래 섹션 내용 복원,Import 주소 복구,메모리 권한 수정,OEP 계산,OEP로 점프 하는 등의 작업을 한다.
- 3.이러한 작업이 끝나면 원래 .text, .data, .rdata 등의 section의 내용이 메모리에 나타난다. 이 때 복원 방식은 패커마다 다르고 대표적인 복원 방식으로는 새로운 메모리에 원래 프로그램을 복원하는 경우도 존재하고 현재 이미지 코드 섹션에 프로그램을 덮어쓰는 방법도 존재한다.
- 4.import 복구 원래 MessageBoxW, CreateFileW, malloc 등을 사용했다면 이 API들의 실제 메모리 주소가 필요하다. 패커는 흔히 LoadLibraryA("kernel32.dll"); ,GetProcAddress(module, "CreateFileW");와 같은 라이브러리를 이용해서 import를 직접 복구한다. 또한 고급 패커들은 이러한 라이브러리의 값들을 해시값으로 암호화시켜서 dll의 export table을 돌면서 해시값이 일치하는 함수를 찾기도 한다.
- 5.이렇게 복원이 끝난 다음에는 패커 stub는 oep 즉 원래의 ep지점으로 이동을 한다.이 시점은 우리가 이전에 알던대로 원래 프로그램이 시작이 된다.

패킹의 증거와 일반 압축 파일과의 차이점

그 다음에 packing 전후의 구조에 대해서 살펴보면 패킹 이전의 원래 프로그램에는 text, rdata, data, pdata, rsrc, reloc와 같은 section 정보가 존재하지만 패킹된 후에 section 정보를 보면 패킹한 프로그램에 따라서 다르지만 UPX0, UPX1, UPX2나 .text, .packed, .stub 또는 무작위 이름을 같은 section이 만들어질 수 있다.

또한 패커에서 실제 메모리 크기와 가상 크기 역시 중요한데 pe 구조에서는 대개 디스크 크기와 메모리 크기를 나타내는 변수가 따로 존재하는 데 SizeOfRawData는 파일에 실제 존재하는 크기고 VirtualSize는 메모리에서 필요한 크기를 나타낸다 이 때 패킹된 프로그램에서는 SizeOfRawData는 작을 수 있다. VirtualSize는 아주 큰 경향이 있어 이를 통해 이 프로그램이 패킹이 되어있는지를 한 번 추론해볼 수 있다. 물론 정상적인 .bss 성격의 데이터도 디스크에는 없고 메모리에서만 공간을 차지할 수 있기 때문에 이것만 보고 프로그램의 패킹 여부를 단정지으면 안된다.

그것 이외에도 패킹된 프로그램의 특징으로는 바이트 분포의 무작위성(엔트로피)가 높은 경향이 있다. 다만 이것도 이미지, 영상, 압축 파일 등의 예외가 있다는 점을 주의해야 한다. 또한 패킹된 프로그램은

import가 매우 적다. 일반적인 프로그램은 굉장히 많은 api를 import할 수 있지만 패킹된 파일은 매우 제한된 api만 import를 할 수 있다. 그 예로는 LoadLibraryA, GetProcAddress, VirtualAlloc, VirtualProtect, ExitProcess 등이 있고 이들을 import하는 이유는 이들을 이용해서 나머지 API를 동적으로 찾을 수 있기 때문이다.

더 강한 패커들은 위에서 말했듯이 import를 아예 없애버리고 PEB를 직접 순회하기도 하기 때문에 import부분에 아무것도 없다면 패킹된 프로그램일 가능성이 아주 높다.

그 외의 증거로는 entry point가 일반 프로그램과 다르게 특이한 영역에 존재하거나 문자열을 확인시 거의 나오지 않거나 색션 영역이 비정상적(rwx 권한이 전부 존재) 다만 최신 패커는 text 영역에 stub를 넣고 거기에 ep를 시작하던가 아니면 권한을 처음엔 쓰기 가능에서 나중엔 실행 가능등으로 바꿀 수 있기 때문에 단일 증거만으로 판단하기 보단 위에 말한 내용들을 체크리스트로 종합적으로 판단하는 것이 좋다.

또한 패킹된 파일과 zip같은 압축 파일에 대해 궁금증을 가질 수도 있는데 이 둘의 차이는 패킹된 파일은 스스로 압축을 풀어 원래 파일을 실행시킬 수 있다면 압축 파일은 windows에서 그 자체로 실행할 수 없다는 차이점이 있다.

언패킹 정의와 방법

언패킹이란 패킹된 프로그램에서 원래의 코드와 구조를 복원하는 과정이다. 크게 동적 언패킹과 정적 언패킹으로 나뉘고 주로 목표는 1. 패커 stub 분석 -> 2. 원래 코드가 복원되는 시점 찾기 -> 3. OEP 찾기 -> 4. 복원된 메모리 이미지 덤프 -> 5. Import Table 재구성 -> 6. PE 헤더와 섹션 정보 수정 -> 7. 정상적인 독립 실행 파일로 저장이다.

또한 메모리에서 원래 프로그램이 보인다고 언패킹이 끝난 건 아니다. 만약 메모리에서 원래 프로그램이 보인다고 해도 import Table이 없음, 섹션의 Raw Offset이 잘못됨, Entry Point가 패커 stub을 가리킴, relocation 정보가 없음, TLS 정보가 손상됨, 예외 처리 정보가 맞지 않음, 메모리 주소 기준 데이터가 파일 구조와 불일치 등의 문제가 있다면 언패킹이 완료되었다고 볼 수 없다.

두 가지 방식의 언패킹 중 정적 언패킹에 대해서 먼저 알아보면 정적 언패킹이란 파일을 실행하지 않고 패커 알고리즘을 분석해서 직접 원본을 복원하는 방식을 취하는 방식으로 마치 리버싱에서 정적 분석과 유사하다. 장점으로는 안전하게 분석이 가능하고 코드로 자동화가 가능하며 패킹 구조를 정확하게 이해할 수 있지만 단점으로는 난이도가 높고, 패커 버전마다 알고리즘이 다를 수 있으며 난독화와 안티디버깅이 강하면 매우 어렵다는 점이 있다.

동적 언패킹이란 프로그램을 디버거에서 실행하여 패커가 스스로 원본 코드를 복원하게 한 뒤 메모리에서 덤프하는 방식으로

x64dbg에서 실행

↓

VirtualAlloc / VirtualProtect 감시

↓

메모리에 코드가 복원되는지 확인

↓

패커 stub 마지막 점프 추적

↓

OEP 도착

↓

메모리 덤프



IAT 복구

등의 절차를 거쳐서 진행한다. 장점은 패커가 복원 작업을 대신 수행한다는 것과 압축 알고리즘을 완전히 이해하지 않아도 가능해서 정적 언패킹보다 난이도가 낮다는 장점이 존재하지만 단점으로는 안티 디버깅에 감지될 수 있고 난독화가 심한 경우 oep를 찾기가 어려우며 Import 재구성이 필요하고 다단계 패킹이면 여러 번 반복해야 한다는 점이 있다.

패커와 pe구조간의 밀접한 관련성

패커는 단순히 코드 바이트만 압축하는 것이 아니다. 실행 파일이 계속 정상적으로 실행되도록 PE 구조 전체를 조정해야 한다. 따라서 pe구조를 자세히 이해하고 있어야 하며 주로 다루는 pe요소는

DOS Header

NT Headers

File Header

Optional Header

Section Table

AddressOfEntryPoint

ImageBase

SizeOfImage

Section RVA

Section Raw Offset

Import Directory

Relocation Directory

TLS Directory

Resource Directory

Section Characteristics

과 같다.

예시를 들면 패커가 새로운 .packed 섹션을 추가했다면 NumberOfSections 증가, 새 Section Header

추가, SizeOfImage 증가 ,AddressOfEntryPoint를 stub으로 변경, 섹션 RVA와 Raw Offset 정렬, 섹션 권한 설정등의 일을 해야 할 수 있다. 이 중 하나라도 잘못된다면 windows loader가 파일의 실행을 거부할 수 있기 때문이다.

간단한 패킹의 예시

원본 코드

```
mov eax, 1
```

```
ret
```

이고 이에 따른 기계어가 B8 01 00 00 00 C3라고 하자 패커가 각 바이트를 0xAA와 XOR한다고 하면

원본:

```
B8 01 00 00 00 C3
```

XOR 0xAA:

```
12 AB AA AA AA 69
```

이기 때문에 패킹된 파일에는 12 AB AA AA AA 69이 저장이 되며 패킹 파일의 stub는

```
lea rsi, [encrypted_code]
```

```
mov rcx, 6
```

decrypt_loop:

```
    xor byte ptr [rsi], 0AAh
```

```
    inc rsi
```

```
    loop decrypt_loop
```

jmp decrypted_code 와 같은 코드로 암호화된 기계어를 다시 언패킹해서 복원을 할 수 있다. 이걸 매우 기본적인 예이며 실제로는 여기에 압축, 여러 단계의 복호화 , API 동적 복구, 안티디버깅,제어 흐름 난독화, 가상 머신, 코드 무결성 검사, 예외 기반 실행 흐름, self-modifying code 등의 다양한 기법이 패커의 종류에 따라서 들어갈 수 있다.

패킹과 프로텍터의 차이

둘은 경계가 겹치지만 보통 다음처럼 구분한다.

일반 패커의 주목적은 파일 크기 감소, 간단한 코드 은닉이라면

프로텍터의 주목적은 리버싱 방지 ,라이선스 보호 ,변조 방지,디버깅 방해이다.

또한 프로텍터는 코드 가상화,안티디버깅,안티덤프,Import 암호화,코드 무결성 검사,제어 흐름 난독화 ,문자열 암호화 등의 기능을 수행하며 그 예시로는 VMProtect, Themida, Enigma Protector등이 있다.
또한 VMProtect 같은 경우 단순히 원본 코드를 압축해 두는 것에서 끝나지 않는다. 일부 함수를 자체 가상 명령어로 변환하고, VM interpreter가 이를 실행하게 할 수 있다.

원래 x86-64 명령어

↓

가상 opcode

↓

VM handler가 해석하여 실행

그래서 단순 메모리 덤프만으로 원래 코드가 완전히 돌아오지 않을 수도 있다.

핵심 용어 정리

용어 의미

Packer 실행 파일을 압축·암호화·변형하는 도구

Packed File 패킹된 실행 파일

Stub 실행 시 원본을 복원하는 작은 코드

EP 현재 PE 헤더가 가리키는 실행 시작점

OEP 패킹 전 원래 프로그램의 진입점

Unpacking 원래 프로그램을 복원하는 작업

Dump 실행 중 메모리 내용을 파일로 저장하는 작업

IAT 프로그램이 사용하는 외부 API 주소 테이블

IAT Rebuild 덤프 후 Import Table을 다시 구성하는 작업

Entropy 데이터의 무작위성 정도

Protector 분석·변조 방지를 중심으로 하는 강한 보호 도구

Self-modifying code 실행 중 자기 코드 영역을 변경하는 코드